"Course Recommendation System"

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

by

Katakamsetty Hasini **:2520030547**

Malkapuram Shravya **:2520030588**

Under the Guidance of

Dr. S. Vinay Kumar
Associate Professor
Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

2026-27

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Declaration

The Project Report entitled **"Course Recommendation System"** is a record of Bonafide work of **Katakamsetty Hasini-2520030547, Malkapuram Shravya-2520030588** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering. The results embodied in this report have not been copied from any other departments/University/Institute.

Katakamsetty Hasini - 2520030547
Malkapuram Shravya - 2520030588

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

CERTIFICATE

This is certify that the mini project based report entitled **"Course Recommendation System"** is a bonafide work done and submitted by **Katakamsetty Hasini – 2520030547, Malkapuram Shravya – 2520030588** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, during the academic year **2026-27.**

Signature of the Guide
(Dr. S. Vinay Kumar)

Signature of the HOD

ACKNOWLEDGEMENT

We are thankful to Dr. P. Venkateshwara Rao, Associate Professor and Head of the Department of Computer Science and Engineering, for providing the facilities and academic environment needed to complete this project. We also thank our faculty members, classmates, and everyone who offered suggestions and assistance during the planning, implementation, testing, and documentation stages.

Finally, we thank our university for giving us the opportunity to apply the concepts learned in our coursework to a practical problem. This project has helped us improve our understanding of web application development, data handling, recommendation techniques, teamwork, and technical documentation.

We would like to express our sincere gratitude to our project guide, Dr. S. Vinay Kumar, Associate Professor, Department of Computer Science and Engineering, for his valuable guidance, encouragement, and support throughout the development of our project, “Course Recommendation System.” His suggestions helped us understand the project requirements and organize our work in a systematic manner.

Katakamsetty Hasini - 2520030547

Malkapuram Shravya - 2520030588

Abstract

ABSTRACT

The **Course Recommendation System** is a personalized web-based application developed to assist students in selecting suitable academic and professional courses based on their individual skills, interests, academic performance, career goals, and learning preferences. In the current educational environment, students have access to a large number of courses offered through universities, online learning platforms, training institutes, and other educational providers. Although this wide range of choices provides flexibility, it also creates difficulty for students in identifying courses that are relevant to their abilities and future career objectives. Many students select courses based on popularity, recommendations from others, or incomplete information, which may result in inappropriate course choices, lack of interest, or difficulty in completing the selected course. The proposed system addresses this problem by providing personalized, data-driven course recommendations.

The system allows students to create an individual profile by providing information such as educational background, academic performance, technical and non-technical skills, areas of interest, preferred difficulty level, learning preferences, and career objectives. The collected information is processed by the recommendation engine and compared with the attributes of available courses. Each course in the system contains details such as course name, description, category, required skills, prerequisites, difficulty level, duration, and career relevance. Based on the similarity between the student's profile and the course requirements, the system calculates a **matching score** for each course. The courses are then ranked according to their scores, allowing students to easily identify the most relevant options.

An important feature of the proposed system is its ability to provide not only course recommendations but also the **reason behind each recommendation**. For example, a course may be recommended because it matches the student's existing programming skills, preferred difficulty level, area of interest, or selected career goal. This explanation-based approach improves transparency and helps students understand why a particular course is suitable for them. The system also provides a personalized dashboard where students can view recommended courses, matching percentages, course descriptions, prerequisites, required skills, duration, and career-related information. Students can search for courses, apply filters based on category or difficulty, view detailed course information, and save courses for future reference.

The proposed system can be implemented using **Python and Flask** for backend development, while **HTML, CSS, and JavaScript** can be used to develop an interactive and user-friendly frontend. **MySQL** can be used as the database for storing student profiles, course information, skills, preferences, and saved courses. **Pandas and NumPy** can be used for data preprocessing, manipulation, and numerical calculations. **Scikit-learn** can be incorporated to implement machine learning and recommendation techniques for improving the accuracy of course matching. Depending on the available dataset and project requirements, the recommendation engine can use a content-based or similarity-based approach in which student attributes are compared with course attributes to generate personalized recommendations.

The system also includes an **administrator module** through which authorized administrators can manage student and course information. Administrators can add new courses, update existing course details, remove outdated courses, manage required skills and prerequisites, and maintain the overall course database. This ensures that the recommendation system works with updated and relevant information. Proper user authentication and database management can also be implemented to protect student information and provide secure access to the system.

The Course Recommendation System can significantly reduce the time and effort required for students to search through large numbers of courses. It provides a systematic approach to course selection by considering multiple factors rather than relying on a single criterion. By connecting a student's current capabilities with their future career objectives, the system can also help students identify areas where additional skills or knowledge may be required. In future versions, the system can be enhanced with advanced machine learning algorithms, skill-gap analysis, personalized learning paths, job-role recommendations, course completion tracking, student feedback, and AI-based conversational assistance.

Overall, the Course Recommendation System provides an intelligent, personalized, and career-oriented solution for informed course selection. By combining student-specific information with course attributes and recommendation techniques, the proposed platform aims to make course discovery more efficient, relevant, transparent, and accessible. It can support students in making better academic and career decisions while helping educational institutions organize and present course information effectively.

Table of Contents

			Page No.
	Acknowledgement		
	Abstract		
	Table of Contents		
	List of Figures		
1	Introduction	1.1 Overview 1.2 Problem statement 1.3 Scope of the project 1.4 Objectives	5-15
2	System Requirements	2.1 Functional Requirements 2.2 Non-Functional Requirements	16-21
3	System Design	3.1 High-level Architecture Diagram 3.2 Description of each layer	33-36
4	Implementation	4.1 Module-wise implementation 4.2 Front-end logic (UI rendering, state management) 4.3 API integration 4.4 …….	38-47
5	Results		56-64
6	Challenges & Limitations	6.1 Issues faced during development 6.2 Solutions applied 6.3 Limitations of the System	74-82
7	Conclusion & Future Work	7.1 Conclusion 7.2 Future Work 7.3 Skills learned during development	89-97
	References	- Books, tutorials, APIs, documentation sites used	98-99

List of Figures

1.1 High-Level Architecture of the Course Recommendation System

1.2 Components of the Course Recommendation System

1.3 Flow Chart of the Course Recommendation System

1.4 Phases of the Course Recommendation System

1.5 Student Profile and Preference Analysis

1.6 Course-Student Matching and Recommendation Process

1.7 Course Recommendation Matching Score Model

1. INTRODUCTION

1.1 Overview

The Course Recommendation System is a web-based application that helps students discover courses suited to their skills, interests, academic background, preferred difficulty level, and career goals. Students create a profile and choose their preferences. The system compares these details with the information available for each course, calculates a matching score, and displays the highest-ranked courses. The recommendation result can include the course description, required skills, prerequisites, duration, difficulty level, and career relevance. Figure 1.1 represents the high-level flow from student input to personalized recommendations.

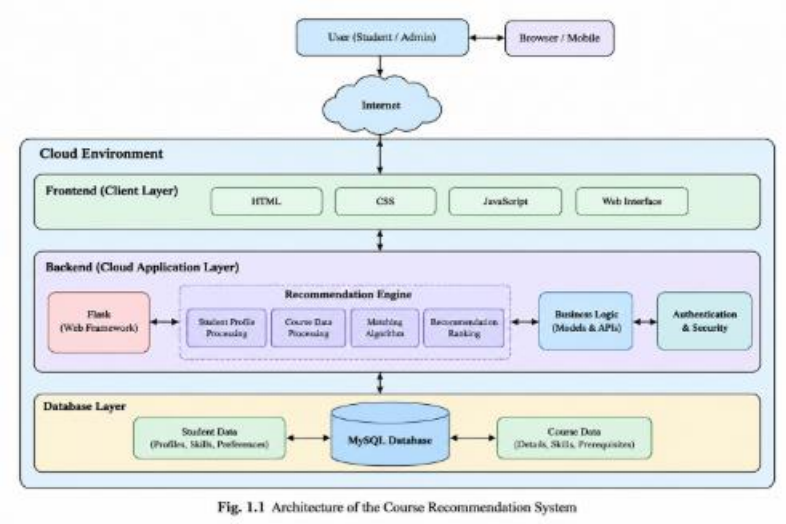


Fig. 1.1 Architecture of the Course Recommendation System

Fig. 1.1 High-Level Architecture of the Course Recommendation System

1.2 Problem Statement

Students often face difficulties while selecting courses because:

- There are many courses available.
- Students may not know which course matches their skills.
- Course prerequisites may not be clear.
- Students may select courses based only on popularity.
- Students may not understand the career relevance of a course.
- Manual course selection takes time.
- Students have different interests and career goals.

Therefore, there is a need for an intelligent system that can analyze student profiles and recommend suitable courses automatically.

1.3 Scope of the Project

The scope of the Course Recommendation System is to provide a personalized platform for discovering academic and professional courses. The system focuses on collecting student preferences, maintaining a course catalogue, comparing student and course attributes, ranking courses, and presenting recommendations in an understandable form. It is intended as a decision-support tool; the final course choice remains with the student.

The student module allows a user to create a profile containing educational background, academic performance, skills, interests, preferred learning style, difficulty preference, and career objective. Students can browse available courses, search by keyword, filter by category or difficulty, open course details, view matching scores, and save courses for later reference.

The administrator module supports adding, updating, viewing, and removing course records. A course record may include the title, description, category, required skills, prerequisites, difficulty, duration, and career relevance. Keeping this catalogue updated is important because recommendations depend on the accuracy and completeness of course information.

The proposed implementation can use Python and Flask for backend request handling, HTML/CSS/JavaScript for the user interface, and MySQL for persistent storage. Pandas and NumPy can support data preparation and calculations, while scikit-learn can provide text vectorization and similarity calculations. The project scope does not require a complex deep-learning model; a content-based or

similarity-based approach is suitable for a clear and explainable prototype.

1.4 Objectives

The main objective is to develop a user-friendly system that recommends courses based on a student's profile and learning goals. The specific objectives are:

- To collect student information such as skills, interests, academic background, and career goals.
- To maintain a structured catalogue of courses and their attributes.
- To compare student preferences with course descriptions, required skills, prerequisites, and categories.
- To calculate a matching score and rank courses from most relevant to least relevant.
- To explain why a course has been recommended, such as matching a skill, interest, or career goal.
- To provide search and filtering options so students can explore courses conveniently.
- To allow authorized administrators to manage and update course information.
- To store student profiles, course records, and saved courses in a database.
- To apply basic data-processing and recommendation concepts in a practical web application.

2. SYSTEM REQUIREMENTS

This section describes the functional and non-functional requirements for the Course Recommendation System. The proposed application consists of a student interface, recommendation logic, administrator functions, and a data-storage component. The technologies described below are proposed implementation choices based on the project abstract.

2.1 Functional Requirements

1. Student Registration and Login

- The system should allow students to create an account and log in with valid credentials.
- The system should associate a student profile with the signed-in user.
- Users should be able to update their profile and preferences when their skills or goals change.

2. Student Profile Management

- The profile should capture educational background and academic performance where provided.
- Students should be able to select technical and non-technical skills, areas of interest, career goals, preferred difficulty, and learning preferences.
- The system should validate required fields and display helpful messages when information is missing.

3. Course Catalogue Management

- The system should maintain course name, description, category, skills, prerequisites, difficulty, duration, and career relevance.
- Students should be able to view course details and search for courses by title or keyword.
- Students should be able to filter courses by category, difficulty level, or other available attributes.

4. Personalized Recommendation

- The system should compare student-profile attributes with course attributes.
- The recommendation engine should calculate a matching score for each eligible course.
- Courses should be ranked by matching score and displayed in descending order.
- The system should show a brief reason for each recommendation, such as a match with the student's selected skills or career goal.
- If no strong match is found, the system should still show available courses or suggest that the student update profile preferences.

5. Saved Courses and Dashboard

- Students should be able to save or bookmark courses for future reference.
- The dashboard should display recommended courses, matching percentages or scores, and key course details.
- The system should allow users to revisit saved courses after logging in again.

6. Administrator Functions

- Authorized administrators should be able to add, edit, view, and remove course records.
- The system should validate course details before saving them.
- Updated course information should be used in later recommendation requests.

7. Data Storage

- The database should store student profiles, course information, preferences, and saved-course records.
- The application should retrieve stored information when the user returns to the system.

2.2 Non-Functional Requirements

1. Usability

The interface should use clear labels, readable text, consistent navigation, and simple forms. Recommendations should be easy to compare, and the reason for a recommendation should be understandable to a student with limited technical knowledge.

2. Performance

The system should process a student profile and rank the available courses within a reasonable time for the size of the project dataset. Course information should be loaded efficiently, and unnecessary repeated calculations should be avoided where possible.

3. Reliability and Accuracy

The system should return consistent results for the same profile and course catalogue when the recommendation method and settings have not changed. Input validation should prevent incomplete or incorrectly formatted records from being stored.

4. Security and Privacy

The application should restrict profile information to authorized users and protect administrator functions. Passwords should be stored using a secure password-hashing method rather than plain text.

Only the student information needed for recommendations should be collected, and database access should be controlled.

5. Maintainability

The application should separate user-interface routes, recommendation logic, database operations, and validation into manageable modules. This structure makes it easier to add new course attributes or improve the recommendation method without rewriting the complete system.

6. Scalability

The design should allow additional courses, skills, categories, and student profiles to be added as the catalogue grows. For a larger deployment, database indexing, pagination, and more efficient recommendation calculations may be introduced.

7. Compatibility

The web application should work in modern browsers and should be designed to remain usable on common laptop and mobile screen sizes. Python, Flask, and MySQL must be installed and configured in the deployment environment.

3. SYSTEM DESIGN

The system design describes how students, the web interface, the recommendation engine, and the database work together. A student submits profile details or updates preferences through the frontend. Flask receives the request, validates the input, retrieves relevant course records from MySQL, and passes the data to the recommendation module. The module processes the attributes, calculates similarity or matching scores, ranks the courses, and returns the results for display.

3.1 High-Level Architecture Diagram

Student / Administrator → Web Interface (HTML, CSS, JavaScript) → Flask Application → Profile and Course Data Retrieval → Data Preprocessing and Recommendation Engine → Ranked Course Recommendations → Dashboard

Database layer: MySQL stores student profiles, course catalogue records, preferences, and saved courses. The Flask backend connects the interface to the database and recommendation logic. Figure 3.1 can be represented using the above flow when preparing the final diagram.

3.2 Description of Each Layer

1. User Layer

Students use the system to enter profile information, explore courses, receive recommendations, and save interesting options. Administrators maintain course information and ensure that catalogue records remain current.

2. Presentation Layer

The presentation layer is developed using HTML, CSS, and JavaScript. It provides registration and login forms, profile forms, course search and filter controls, course-detail views, a recommendation dashboard, and administrator screens. It displays matching scores and recommendation explanations in a readable format.

3. Application Layer

The application layer uses Flask to handle HTTP requests, validate form data, manage sessions, call database functions, and return pages or results. It applies access checks before allowing users to view personal records or use administrator functions.

4. Data Preprocessing Layer

This layer prepares student and course attributes for comparison. Text fields can be normalized by converting text to a consistent case, removing unnecessary spaces, and combining relevant terms. Missing optional fields should be handled consistently so that they do not create misleading matches.

5. Recommendation Layer

The recommendation layer represents course attributes in a comparable form. A content-based approach can convert course descriptions, skills, interests, and goals into numerical vectors. Cosine similarity or a weighted rule-based score can then estimate how closely each course matches a student profile. The scores are sorted, and the highest-ranked courses are returned.

6. Database Layer

The MySQL database stores user accounts, student profiles, course details, preferences, and saved-course relationships. Suitable keys and relationships should be used to keep records consistent. Database queries should use parameters rather than directly concatenating user input.

4. IMPLEMENTATION

Implementation can be divided into independent modules so that each function can be developed and tested separately. The proposed technology stack is Python and Flask for backend processing, HTML/CSS/JavaScript for the frontend, MySQL for persistent data, and Pandas, NumPy, and scikit-learn for data handling and similarity calculations.

4.1 Module-Wise Implementation

Module 1 – User Registration and Authentication

This module accepts registration details and validates the input. After successful login, the system identifies the user's role and opens the relevant dashboard. Passwords should be hashed before storage, and protected routes should require a valid session.

Module 2 – Student Profile Management

The student enters education details, skills, interests, preferred difficulty, learning preferences, and career objectives. The backend validates the submitted values and stores the profile. Students can update the profile as their interests or goals change.

Module 3 – Course Catalogue

Each course is represented by a structured record containing a course identifier, name, description, category, required skills, prerequisites, difficulty, duration, and career relevance. The catalogue module supports course listing, detail views, search, filtering, and administrator updates.

Module 4 – Data Preprocessing

Before comparison, the application normalizes textual attributes and prepares the student profile and course records for scoring. Depending on the implementation, selected text fields can be combined and

transformed using a vectorizer such as TF-IDF. Numeric or categorical attributes can be handled separately and combined using a weighted score.

Module 5 – Recommendation Engine

The recommendation engine compares the profile representation with each course representation. For a text-based implementation, cosine similarity can measure the angle between the student vector and a course vector. A larger similarity value indicates a closer match in the represented attributes. The system sorts courses by score and selects the top results.

A simple weighted matching score may also be used: Matching Score = (Skill Match Weight × Skill Match) + (Interest Weight × Interest Match) + (Career Goal Weight × Career Match) + (Difficulty Preference Weight × Difficulty Match). The weights should be chosen consistently and normalized so the displayed score has a clear meaning. If percentages are shown, they should be treated as recommendation scores rather than guarantees of course success.

Module 6 – Recommendation Explanation

For every recommended course, the system identifies matching profile attributes, such as a shared skill, selected interest, or career goal. These explanations help students understand why the course appears in the result list and make the recommendation process more transparent.

Module 7 – Saved Courses

Students can save selected courses to a personal list. The application stores the student identifier and course identifier and retrieves the saved list on later visits. Duplicate saved records should be prevented.

Module 8 – Administrator Management

Administrators can create new course records, update descriptions and prerequisites, change course categories, and remove outdated records. Input validation is required to avoid incomplete catalogue entries. Changes should be reflected in future recommendation requests.

4.2 Front-End Logic and User Interface

The frontend provides forms and views for registration, login, profile editing, course browsing, recommendation results, saved courses, and administration. JavaScript can validate simple fields and improve interaction, while the backend remains responsible for final validation. Recommendation cards should display course title, category, difficulty, duration, matching score, and a short explanation.

4.3 API and Backend Integration

Flask routes connect browser actions to application functions. Typical routes may include registration, login, profile update, course listing, course search, recommendation generation, saved-course operations, and administrator course management. The backend retrieves data from MySQL, calls the recommendation engine, and returns the results to the frontend. Route names depend on the final source-code implementation.

4.4 Database Design

A practical database may include the following tables: Users (user_id, name, email, password_hash, role); StudentProfiles (profile_id, user_id, education, academic_details, interests, career_goal, difficulty_preference); Courses (course_id, title, description, category, skills, prerequisites, difficulty,

duration, career_relevance); and SavedCourses (user_id, course_id, saved_at). The exact fields can be adjusted to match the final implementation.

4.5 Recommendation Workflow

First, the student logs in and completes the profile. Next, the system validates the information and retrieves the course catalogue. The preprocessing component prepares profile and course attributes. The recommendation engine computes matching scores, ranks the courses, and returns the top matches. The dashboard displays the ranked list and explanations, and the student may open a course or save it for later.

5. RESULTS

5.1 Functional Results

The following test cases can be used to evaluate the completed application. The expected results are described here as a testing plan; actual execution screenshots and measured values should be inserted after running the final program.

Test Area | Expected Result

Registration and Login | Valid users can register and sign in; invalid credentials are rejected.

Profile Update | Student preferences are saved and can be retrieved later.

Course Catalogue | Available courses and their details are displayed correctly.

Search and Filter | Results are narrowed according to the entered keyword or selected filters.

Recommendation | Courses are ranked according to the profile-to-course matching method.

Recommendation Explanation | The result shows relevant matching skills, interests, or career goals.

Saved Courses | A student can save a course and view it later; duplicate entries are prevented.

Administrator Update | Authorized administrators can add or update course records.

Input Validation | Missing or invalid input produces a helpful message.

5.2 Recommendation Results and Evaluation

Recommendation quality depends on the course catalogue, profile information, preprocessing choices, and scoring method. For a small prototype, the system can be checked with sample student profiles representing different skills and career goals. For example, a student interested in web development and familiar with HTML, CSS, and JavaScript should rank relevant frontend or full-stack courses above unrelated courses, provided those attributes are present in the catalogue.

Useful evaluation measures include whether the top-ranked courses are relevant to the student's stated goals, whether the results change when profile preferences change, and whether the same input produces consistent rankings. If labelled relevance data is available, precision at K or other recommender-system metrics may be calculated. No numerical accuracy claim should be made unless it has been measured on a defined dataset.

6. CHALLENGES AND LIMITATIONS

6.1 Issues Faced During Development

- Representing different student attributes and course information in a consistent format.

- Handling incomplete profiles and course records without generating misleading scores.
- Choosing suitable weights or similarity measures for different attributes.
- Maintaining correct relationships between user profiles, courses, and saved records.
- Presenting recommendation scores in a way students can understand.
- Testing the recommendation engine with different combinations of skills, interests, and career goals.

6.2 Solutions Applied

- Use structured profile and course fields with input validation.
- Normalize textual attributes and apply consistent preprocessing before comparison.
- Keep recommendation calculations in a separate module so the scoring method can be tested independently.
- Use database keys and constraints to maintain consistent records and prevent duplicate saved courses.
- Display the attributes that contributed to each recommendation instead of showing a score alone.
- Test with several sample profiles and review whether the ranked results are reasonable.

6.3 Limitations of the System

- Recommendations depend on the accuracy and coverage of the course catalogue and student-provided information.
- A content-based prototype may recommend courses similar to those already represented in the profile and may not discover unexpected interests.
- The matching score is an estimate of relevance, not a guarantee of academic success or job placement.
- The quality of recommendations may be limited if no labelled evaluation dataset is available.
- Course prerequisites and external course availability must be updated by the administrator unless an external integration is added.
- The final performance and supported user load depend on the actual implementation and deployment environment.

7. CONCLUSION AND FUTURE WORK

7.1 Conclusion

The Course Recommendation System is designed to help students identify courses that are relevant to their skills, interests, academic background, preferred difficulty, and career goals. It brings student profiles and course information together and uses a recommendation engine to calculate matching scores and rank suitable options. The proposed web application includes profile management, course browsing, search and filtering, recommendation explanations, saved courses, and administrator course management.

The project demonstrates how web development, database management, data preprocessing, and similarity-based recommendation techniques can be combined to address a practical educational problem. By showing why a course is recommended, the system aims to make course selection more transparent and reduce the time students spend exploring unrelated options. The effectiveness of the

system should be confirmed through testing with representative profiles and a well-maintained course catalogue.

7.2 Future Work

- Integrate a larger and regularly updated course dataset from trusted educational providers.
- Improve the recommendation method by combining content-based recommendations with collaborative filtering when sufficient user-interaction data is available.
- Add skill-gap analysis that compares current student skills with the requirements of a selected career role.
- Generate personalized learning paths that arrange courses in a logical prerequisite order.
- Include course ratings, student feedback, and completion tracking to improve future recommendations.
- Add job-role and certification recommendations based on career goals.
- Provide multilingual support, mobile-friendly design, and accessibility improvements.
- Evaluate the system using appropriate recommender metrics and feedback from actual student users.

7.3 Skills Learned During Development

- Planning and documenting a web-based software project.
- Frontend development using HTML, CSS, and JavaScript.
- Backend development concepts using Python and Flask.
- Database design and CRUD operations using MySQL.
- Data preprocessing and attribute representation.
- Similarity calculations and basic recommender-system concepts.
- Input validation, testing, debugging, and error handling.
- Teamwork, technical writing, and project presentation.

REFERENCES

[1] O. A. S. Ibrahim et al., “Revisiting recommender systems: an investigative survey,” *Neural Computing and Applications*, vol. 37, pp. 2145–2173, 2025.

[2] J. Joy and R. V. G. Pillai, “Review and classification of content recommenders in E-learning environment,” *Journal of King Saud University – Computer and Information Sciences*, vol. 34, pp. 7670–7685, 2022.

[3] O. A. S. Ibrahim et al., “Recommender Systems and Recommendation Techniques: A Survey,” 2025.

[4] Y. Koren and R. Bell, “Advances in Collaborative Filtering,” *Recommender Systems Handbook*, Springer, pp. 145–186, 2010.

[5] Scikit-learn Developers, “Cosine Similarity,” *Scikit-learn Documentation*.

[6] Pallets Projects, “Flask Documentation,” *Flask Web Framework Documentation*.